

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 6

Operating Systems

Topic: Inside the xv6 File System

mystat, the Limits of fs.h and a Recursive find

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Short Theory **4**
1.1 The Layout of the xv6 Disk 4
1.2 Inodes and Directories 4
1.3 The stat System Call 4

2 Read What xv6 Shows You **5**

3 Write mystat **7**
3.1 Implementation 7
3.2 Watching Hard Links at Work 8

4 Measure the Limits **11**

5 Write find **13**
5.1 Implementation 13
5.2 Testing on a Fresh Disk 15

6 Stretch Task: find with Inode Numbers **16**

Conclusions **17**

INTRODUCTION

The purpose of this laboratory work is to look at the same file system ideas as in Laboratory Work No. 5 – inodes, directories as tables of names, hard links, a journal – but this time inside xv6, where the whole file system is a few hundred readable lines of C. The work has four steps: reading what the xv6 `ls` shows, writing a `mystat` program that prints the fields of `struct stat`, measuring the limits written in `kernel/fs.h`, and writing `find`, a program that walks the directory tree recursively (the `find` exercise of the MIT 6.1810 util lab).

I used the same `xv6-riscv` tree as in the previous labs. Every new program is a file `user/NAME.c` plus a line `$U/_NAME\` in the `UPROGS` list of the `Makefile`. The submitted files are `mystat.c`, `find.c` and the `Makefile`.

1 SHORT THEORY

1.1 The Layout of the xv6 Disk

The xv6 disk image `fs.img` is an array of blocks of `BSIZE` = 1024 bytes. The first blocks are *metadata*: block 0 is the boot block (unused), block 1 is the superblock that describes the layout, then come the blocks of the *log* (the journal that makes multi-block updates crash-safe), the blocks with the on-disk inodes, and the free-block bitmap. All the remaining blocks hold file data. The tool `mkfs` builds this image at compile time and prints the layout.

1.2 Inodes and Directories

An on-disk inode (`struct dinode`) stores the type of the file, the device numbers, the link count, the size and the addresses of the data blocks: `NDIRECT` = 12 direct block numbers and one *indirect* block, which itself contains block numbers. A directory is a file of type `T_DIR` whose content is an array of `struct dirent`: a 2-byte inode number and a name of at most `DIRSIZ` = 14 characters. An entry with inode number 0 is a free slot. Exactly as on Linux, the name lives only in the directory and the inode does not know its own name.

1.3 The stat System Call

`stat(path, &st)` (and `fstat(fd, &st)`) copies the metadata of an inode into a `struct stat` defined in `kernel/stat.h`: `dev`, `ino`, `type`, `nlink` and `size`. Programs like `ls` and `find` use it to learn whether a name is a file, a directory or a device.

2 READ WHAT XV6 SHOWS YOU

After `make clean ; make qemu` I listed the root directory and created an empty directory (Figure 1):

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35560
forktest       2 5 17464
grep           2 6 40176
init           2 7 36016
kill           2 8 35496
ln             2 9 35312
ls             2 10 38856
mkdir          2 11 35544
rm             2 12 35528
sh             2 13 58608
stressfs       2 14 36416
usertests      2 15 202984
grind          2 16 51792
wc             2 17 37632
zombie         2 18 34896
logstress      2 19 37480
forphan        2 20 36344
dorphan        2 21 35808
sync           2 22 34960
sleep          2 23 35280
args           2 24 35280
console        3 25 0
$ mkdir a
$ la a
exec la failed
$ ls a
.              1 26 32
..             1 1 1024
$
```

Figure 1: The root directory of xv6 and a new, empty directory

Every line is: name, type, inode number, size in bytes. To decode the type I opened `kernel/stat.h`:

```c
#define T_DIR    1 // Directory
#define T_FILE   2 // File
#define T_DEVICE 3 // Device

struct stat {
  int dev;     // File system's disk device
  uint ino;    // Inode number
  short type;  // Type of file
  short nlink; // Number of links to file
  uint64 size; // Size of file in bytes
};
```

Listing 1: kernel/stat.h

Observations

1. What do types 1, 2 and 3 mean (`kernel/stat.h`)?

Type 1 is `T_DIR`, a directory; type 2 is `T_FILE`, a regular file (`README` and all the programs); type 3 is `T_DEVICE`, a device file – here only `console`, whose size is 0 because its data comes from the console driver and not from disk blocks.

2. Which inode number does the root directory have, and why are `.` and `..` the same there?

The root directory has inode number 1 (`ROOTINO` in `kernel/fs.h`; inode 0 is never used, because 0 marks a free directory entry). Both `.` and `..` show inode 1: `.` always points to the directory itself, and the root has no parent, so `mkfs` makes its `..` point back to itself as well. That is why `cd ..` in `/` stays in `/`.

3. An empty directory `a` has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does `a` contain?

One `struct dirent` is 16 bytes: a `ushort inum` (2 bytes) and `char name[DIRSIZ]` (14 bytes). An "empty" directory still contains two entries, $2 \times 16 = 32$ bytes: `.` pointing to itself (the inode printed on the `.` line of `ls a` in Figure 1) and `..` pointing to its parent, the root (inode 1).

3 WRITE MYSTAT

3.1 Implementation

The skeleton had two TODOs: translate the numeric type into a word, and call `stat()` for every argument, printing an error on descriptor 2 if it fails and continuing with the next argument.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t){
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;

  if(argc < 2){
    fprintf(2, "usage: mystat <path>...\n");
    exit(1);
  }

  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Listing 2: user/mystat.c

The program was added to `UPROGS`:

```
sergiu@debian:~/xv6-riscv$ grep -n '_mystat|_find' Makefile
sergiu@debian:~/xv6-riscv$ tail -12 Makefile
        @echo $(GDBPORT)

QEMU_VERSION := $(shell $(QEMU) --version | head -n 1 | sed -E 's/^QEMU emulator version ([0-9]+\.[0-9]+)\..*/\1/')
check-qemu-version:
        @if [ "$(shell echo "$(QEMU_VERSION) >= $(MIN_QEMU_VERSION)" | bc)" -eq 0 ]; then \
                echo "ERROR: Need qemu version >= $(MIN_QEMU_VERSION)"; \
                exit 1; \
        fi

.PHONY: fmt
fmt:
        clang-format -i $(wildcard kernel/*.[ch] user/*.[ch] mkfs/*.c)
sergiu@debian:~/xv6-riscv$
```

Figure 2: The new lines at the end of UPROGS

3.2 Watching Hard Links at Work

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$ clear
exec clear failed
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
$ mystat nothere
mystat: cannot stat nothere
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ >mystat r2
exec r2 failed
$  mystat r2
exec  failed
$ cat r2 | wc
48 336 2441
$
```

Figure 3: mystat before and after ln and rm

(The root directory reports `links 1`: `mkfs` gives the root inode one link, and in this freshly rebuilt image no subdirectory points back to it yet. After a `mkdir a` the count becomes 2, because `create()` in `kernel/sysfile.c` does `dp->nlink++` for every new subdirectory, to account for its `..` entry.)

Observations

1. Paste the outputs after `ln` and after `rm`. What changed and what did not?

After `ln README r2` both names report *the same* inode 2, and the link count went from 1 to 2. After `rm README` the name `README` disappeared and `r2` reports the same inode with links 1. What did **not** change: the inode number, the type and the size (2441 bytes) – and the content, as `cat r2 | wc` shows the full README. `ln` and `rm` only add and remove directory entries and adjust `nlink`; the inode and its data blocks were never touched.

2. Find the line in `kernel/sysfile.c` (`sys_unlink`) that decrements the link count. When are the data blocks freed?

```
235   memset(&de, 0, sizeof(de));
236   if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
237     panic("unlink: writei");
238   if (ip->type == T_DIR) {
239     dp->nlink--;
240     iupdate(dp);
241   }
242   iunlockput(dp);
243
244   ip->nlink--;
245   iupdate(ip);
246   iunlockput(ip);
```

Listing 3: kernel/sysfile.c, end of sys_unlink

The decrement is `ip->nlink--;` (line 244 in my tree). Before it, the directory entry is overwritten with zeros, which turns it into a free slot (`inum == 0`). `sys_unlink` does **not** free any block. That happens in `iput()` in `kernel/fs.c`, called through `iunlockput(ip)`: when the last in-memory reference is dropped (`ip->ref == 1`) *and* the link count is 0, `iput` calls `itrunc(ip)`, which frees all direct and indirect data blocks, and then marks the inode free. So the blocks are freed only when no name and no open file refers to the inode any more – the same rule as on Linux.

3. xv6 has no symbolic links and no permissions. Look at `struct dinode` in `kernel/fs.h`: which fields does an xv6 inode have?

```
1 struct dinode {
2   short type;              // File type
3   short major;             // Major device number (T_DEVICE only)
4   short minor;             // Minor device number (T_DEVICE only)
5   short nlink;             // Number of links to inode in file system
6   uint size;               // Size of file (bytes)
7   uint addrs[NDIRECT + 1]; // Data block addresses
8 };
```

Listing 4: kernel/fs.h, the on-disk inode

An xv6 inode has only six fields: `type`, `major` and `minor` (for devices), `nlink`, `size` and `addrs[13]` (12 direct block numbers plus one indirect block). Compared with ext4 it has no owner (UID/GID), no permission bits, no timestamps and no symlink type. Its size is $4\times2+4+13\times4 = 64$

bytes, so 16 inodes fit in one 1024-byte block.

4 MEASURE THE LIMITS

`kernel/fs.h` holds the constants that describe the geometry of the file system, and the `mkfs` tool prints the layout it writes into `fs.img` at build time (Figure 4):

```
sergiu@debian: ~/xv6-riscv
sergiu@debian:~/xv6-riscv$ grep -n "define BSIZE|define NDIRECT|define NINDIRECT|define DIRSIZ" kernel/fs.h
sergiu@debian:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*/*.o */*.d */*.asm */*.sym \
kernel/kernel fs.img \
mkfs/mkfs .gdbinit \
        user/usys.S \
user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressf
s user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync user/_sleep user/_args
user/_mystat
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
sergiu@debian:~/xv6-riscv$
```

Figure 4: The constants in kernel/fs.h and the layout printed by mkfs

The same constants limit the names: `DIRSIZ` is 14, so a longer name is silently cut when the directory entry is written (Figure 5).

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ mkdir averyveryverylongname
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35560
forktest       2 5 17464
grep           2 6 40176
init           2 7 36016
kill           2 8 35496
ln             2 9 35312
ls             2 10 38856
mkdir          2 11 35544
rm             2 12 35528
sh             2 13 58608
stressfs       2 14 36416
usertests      2 15 202984
grind          2 16 51792
wc             2 17 37632
zombie         2 18 34896
logstress      2 19 37480
forphan        2 20 36344
dorphan        2 21 35808
sync           2 22 34960
sleep          2 23 35280
args           2 24 35280
mystat         2 25 36424
console        3 26 0
averyveryveryl 1 27 32
$
```

Figure 5: A file name longer than DIRSIZ

Observations

1. What does `ls` show for `averyveryverylongname`, and which constant explains it?

`ls` shows `averyveryveryl`: the name was cut to its first 14 characters. The constant is `DIRSIZ = 14`: a `struct dirent` has room for exactly 14 bytes of name. `skipelem()` and `dirlink()` in `kernel/fs.c` silently copy only `DIRSIZ` bytes, and when the name is exactly 14 characters long there is no terminating zero in the entry – this is why `ls` copies the name into a buffer and adds `p[DIRSIZ] = 0` itself.

2. Compute the largest file xv6 can hold.

$$\begin{aligned} \text{NINDIRECT} &= \frac{\text{BSIZE}}{\text{sizeof(uint)}} = \frac{1024}{4} = 256 \text{ block numbers,} \\ \text{MAXFILE} &= \text{NDIRECT} + \text{NINDIRECT} = 12 + 256 = 268 \text{ blocks,} \\ \text{max size} &= 268 \times 1024 = 274\,432 \text{ bytes} = 268 \text{ KiB.} \end{aligned}$$

A file of more than 268 KiB cannot be written in xv6; `writei()` refuses to go past `MAXFILE * BSIZE`.

3. From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?

Table 1: Layout of fs.img (blocks of 1024 bytes)

Region	**Blocks**	**Where it comes from**
boot + superblock	2	blocks 0 and 1
log	31	1 header + `LOGBLOCKS` = 3× `MAXOPBLOCKS` = 30
inodes	13	`NINODES/IPB` +1 = 200/16 + 1
bitmap	1	2000 bits fit in one block of 8192 bits
metadata (nmeta)	47	2 + 31 + 13 + 1
data	1953	2000 − 47
total	**2000**	`FSSIZE` in `kernel/param.h`

The whole disk has 2000 blocks (about 2 MB): 31 for the log, 13 for the inodes, 1 for the bitmap and 1953 for data – exactly what the `nmeta` line in Figure 4 reports.

5 WRITE FIND

5.1 Implementation

I started from `user/ls.c`, which already shows how to open a directory, read `struct dirent` entries one by one, build the full path in a buffer and call `stat` on it. The changes are:

- entries with `inum == 0` (free slots) are skipped;
- `.` and `..` are never followed, otherwise the recursion would never end;
- the last component is compared with the wanted name using `strcmp`, and the full path is printed on a match;
- if the entry is a directory, `find` calls itself on the new path;
- every opened directory is closed before returning, because a process may have only 16 open files (`NOFILE`) and a deep recursion would otherwise run out of descriptors;
- the optional third argument `-n` (the stretch task) also prints the inode number of each match.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

// find <dir> <name> [-n]
// prints every path under <dir> whose last component is <name>;
// with -n it also prints the inode number of each match.

int show_inode = 0;

// return a pointer to the last component of path
char *
lastname(char *path)
{
  char *p;

  for(p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  return p + 1;
}

void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
```

```
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type != T_DIR){
    // find was started on a file: only compare its own name
    if(strcmp(lastname(path), name) == 0){
      if(show_inode)
        printf("%s %d\n", path, st.ino);
      else
        printf("%s\n", path);
    }
    close(fd);
    return;
  }

  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }
  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)                      // free directory slot
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;                           // never walk back up the tree
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(strcmp(p, name) == 0){
      if(show_inode)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }
    if(st.type == T_DIR)
      find(buf, name);                    // recurse into subdirectories
  }
  close(fd);                              // only 16 open files per process
}

int
main(int argc, char *argv[])
{
  if(argc < 3 || argc > 4 || (argc == 4 && strcmp(argv[3], "-n") != 0)){
```

```
91      fprintf(2, "usage: find <dir> <name> [-n]\n");
92      exit(1);
93    }
94    if(argc == 4)
95      show_inode = 1;
96    find(argv[1], argv[2]);
97    exit(0);
98  }
```

Listing 5: user/find.c

5.2 Testing on a Fresh Disk

```
init: starting sh
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$ find . nothing
$ find
usage: find <dir> <name> [-n]
$
```

Figure 6: find on a fresh disk

The output (Figure 6) contains exactly the three expected paths. The order follows the order of the entries in each directory: `a` was created before `r2` in the root, so the subtree of `a` is printed first.

6 STRETCH TASK: FIND WITH INODE NUMBERS

```
$ find . r2 -n
./a/b/r2 30
./a/r2 31
./r2 3
$ mystat cat r2
cat: file, inode 3, links 2, size 36728 bytes
r2: file, inode 3, links 2, size 36728 bytes
$
```

Figure 7: find . r2 -n and the inodes of the two names of cat

Only `./r2` is the same file as `cat`: `find . r2 -n` prints inode 3 for it, and `mystat` shows that this inode has 2 links. `./a/r2` (inode 31) and `./a/b/r2` (inode 30) are new, independent files created by `echo`, which received fresh inodes (Figure 7). Running `./r2` would in fact start the `cat` program.

CONCLUSIONS

In this laboratory work I explored the xv6 file system from the user side and from the source code. Decoding the output of `ls` showed that an xv6 directory is literally an array of 16-byte entries, that the root has inode 1 and is its own parent, and that even an empty directory holds `.` and `..`. Writing `mystat` made the content of `struct stat` visible and allowed me to watch hard links: `ln` and `rm` only change directory entries and `nlink`, while the inode and its data stay the same. Reading `sys_unlink` and `iput` confirmed the rule I had already observed on Linux in Laboratory Work No. 5: blocks are freed only when the link count is zero and nobody uses the inode any more.

Measuring the limits from `kernel/fs.h` showed how small xv6 deliberately is: names of at most 14 characters, files of at most 268 KiB, and a 2000-block disk of which 47 blocks are metadata, including a 31-block log. Finally, by turning `ls` into a recursive `find` I practised walking the directory tree, skipping free slots and `./..`, and closing every descriptor to stay within the per-process limit. The stretch task with inode numbers showed once more that a name is only a pointer to an inode: `./r2` and `cat` are two names of the same file.